

MLOps POC Guideline

A 2–3 day, sub-\$10 proof of concept to onboard DevOps engineers to MLOps

Audience: DevOps engineers, no prior MLOps exposure **Out of scope:** Existing ML engineering team / production ML systems **Goal:** Give the DevOps team hands-on muscle memory for the four MLOps pillars — **CI, CD, CT, CM** — using tools that mirror what they already know, at near-zero cost.

1. Why this design

DevOps already owns CI/CD, IaC, containers, and monitoring. MLOps is the same discipline applied to a moving target: **data and models change, not just code**. The POC is built so every new MLOps concept is introduced as "the thing you already do, plus one twist":

DevOps concept	MLOps twist	POC tool
Code versioning (Git)	Data & model versioning	Git + DVC
CI (lint/test/build)	+ data & model validation tests	GitHub Actions
CD (deploy artifact)	Deploy a model artifact/container	GitHub Actions → AWS Lambda
— (new concept)	CT: retraining is itself a pipeline step	GitHub Actions scheduled/triggered job
Monitoring (uptime, latency)	+ data drift / prediction quality	CloudWatch + Evidently AI

This keeps the learning curve to one new idea at a time rather than a full ML platform.

2. Scope decision: keep the model boring, make the pipeline the star

Since the goal is teaching **pipeline discipline**, not ML skill, the model itself should be trivial to train (seconds, CPU-only, no GPU cost) so the team's attention stays on CI/CD/CT/CM.

Recommended POC use case: A binary classifier on the **Telco Customer Churn dataset** (IBM, 7,043 rows × 21 columns) using **scikit-learn**. It's a mix of categorical (contract type, payment method, internet service) and numeric (tenure, monthly charges) features — while still training in seconds on a laptop.

Recommended open-source model: `scikit-learn RandomForestClassifier` or `LogisticRegression`.

- 100% open source, no license concerns
- Trains in <5 seconds on any laptop or CI runner — no GPU, no special compute
- Small enough (KBs) to package in a container and deploy on Lambda for pennies

Optional stretch (skip unless time allows): Swap the model for a pre-trained Hugging Face model such as `distilbert-base-uncased-finetuned-sst-2-english` for text sentiment classification, to show the team what deploying a "real" pretrained model looks like. Not recommended for the first pass given the 2–3 day window — it adds container size, memory, and cold-start considerations that distract from the pipeline lessons.

3. Tool stack (mapped to your budget & CI tool)

Pillar	Tool	Cost
Source & data versioning	Git (GitHub) + DVC	Free
Artifact/data storage	Amazon S3	Free tier (5GB) — fractions of a cent
Experiment tracking	MLflow (local file-based, no server)	Free
CI (test, lint, validate)	GitHub Actions (existing tool)	Free (2,000 min/mo on free plan)
Containerization	Docker + Amazon ECR	Free tier: 500MB storage/mo, ~\$0.10/GB after
CD (deploy)	AWS Lambda (container image support) + Function URL	Free tier: 1M requests/mo

CT (retrain trigger)	GitHub Actions scheduled workflow / manual dispatch	Free
CM (monitoring)	Amazon CloudWatch Logs + Evidently AI (open source drift reports)	Free tier: 5GB logs/mo
IaC (optional, if time permits)	Terraform (open source) for the Lambda + IAM role	Free

Expected total AWS spend for the POC: \$0–2, comfortably inside your \$10 ceiling. Lambda, S3, and CloudWatch free tiers cover this workload many times over; ECR free tier (500MB) is more than enough for one small image, so storage cost stays near \$0. The only realistic cost driver is if old image versions pile up in ECR or S3 storage isn't cleaned up afterward — flag this for teardown at the end.

4. Repo structure

```

mlops-poc /
├── data/           # raw + processed data (tracked via DVC, not Git)
├── src/
│   ├── prepare_data.py  # downloads raw Telco Churn CSV, basic cleaning, writes to data/
│   ├── train.py         # trains model, logs to MLflow
│   ├── validate.py      # data & model quality checks (the "new" CI step)
│   ├── predict.py       # inference handler for Lambda
│   └── monitor.py       # Evidently AI drift check script
├── tests/
│   └── test_model.py    # unit tests: schema checks, accuracy threshold
├── Dockerfile
├── dvc.yaml         # pipeline stages: prepare -> train -> evaluate
├── .github/workflows/
│   ├── ci.yml         # lint + test + validate on every PR
│   ├── train-deploy.yml  # CT + CD: retrain, build image, push, deploy
│   └── monitor.yml     # scheduled CM: drift report
└── README.md

```

5. Day-by-day plan

Day 1 — Foundations (Data + Model + Experiment tracking)

1. Create the GitHub repo, connect DVC with an S3 remote (`dvc remote add`).

2. Write `prepare_data.py`: download the Telco Customer Churn CSV, do minimal cleaning (handle blank `TotalCharges` values, encode categorical columns), write the result to `data/telco_churn.csv`. Register it as the first DVC pipeline stage (`dvc stage add -n prepare ...`) so raw → processed data is reproducible, not a one-off manual download.
3. Write `train.py`: load the processed Telco Churn data → train `RandomForestClassifier` → log params/metrics/model to **MLflow** (local `mlruns/` folder is enough, no server needed).
4. Write `tests/test_model.py`: assert accuracy > a fixed threshold (e.g., 75%), assert expected input schema — this is the "unit test" equivalent for ML.
5. Commit code to Git, commit data/model pointers via DVC, push data to S3.

Outcome: the team sees data and models can be versioned and tested exactly like code.

`src/prepare_data.py`:

```
import pandas as pd
```

```
RAW_URL = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"
```

```
OUTPUT_PATH = "data/telco_churn.csv"
```

```
CATEGORICAL_COLS = [
    "gender", "Partner", "Dependents", "PhoneService", "MultipleLines",
    "InternetService", "OnlineSecurity", "OnlineBackup", "DeviceProtection",
    "TechSupport", "StreamingTV", "StreamingMovies", "Contract",
    "PaperlessBilling", "PaymentMethod",
]
```

```
def prepare():
```

```
    df = pd.read_csv(RAW_URL)
```

```
    # Minimal cleaning - keep it simple, this isn't the point of the POC
```

```
    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
```

```
    df["TotalCharges"] =
```

```
    df["TotalCharges"].fillna(df["TotalCharges"].median())
```

```
    df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0})
```

```
    # Simple label-encode the categoricals so scikit-learn can consume them
    directly
```

```
    for col in CATEGORICAL_COLS:
```

```
        df[col] = df[col].astype("category").cat.codes
```

```
    keep_cols = CATEGORICAL_COLS + [
```

```

        "SeniorCitizen", "tenure", "MonthlyCharges", "TotalCharges", "Churn",
    ]
    df = df[keep_cols]

    df.to_csv(OUTPUT_PATH, index=False)
    print(f"Wrote {len(df)} rows to {OUTPUT_PATH}")

if __name__ == "__main__":
    prepare()

```

□ Run it once with `python src/prepare_data.py`, then `dvc add data/telco_churn.csv` (or let the `dvc stage` definition handle tracking automatically on `dvc repro`) and push to the S3 remote with `dvc push`.

Note the extra richness vs. a Titanic-style dataset: 15 categorical columns instead of one, giving `prepare_data.py` a non-trivial encoding step and giving the later drift-detection work (Section 10.2) more feature distributions worth watching.

Day 2 — CI/CD (GitHub Actions → Lambda)

1. `ci.yml`: on every PR — lint, run `pytest`, run `validate.py` (data drift/schema gate). Fail the build if the model doesn't meet the accuracy bar — this is the CI "quality gate" DevOps already understands.
2. Build a small Docker image wrapping `predict.py` as a Lambda handler. Create an ECR repository (`aws ecr create-repository`) to hold it.
3. `train-deploy.yml`: on merge to `main` — retrain model (**CT**), authenticate to ECR, build and push the image, update the Lambda function to point at the new image URI via AWS CLI/GitHub Actions AWS credentials action (**CD**). Use a lifecycle policy on the ECR repo to auto-expire old untagged images so storage never grows unbounded.
4. Expose the Lambda via a **Function URL** (simplest, avoids API Gateway cost/complexity) with `AuthType: AWS_IAM` (SigV4-signed requests only — avoids leaving a public endpoint live even briefly) and test with a signed request.
5. Add a **smoke-test step** at the end of `train-deploy.yml`: after deploying, send one known request to the Function URL and assert the response shape/status. Same pattern as a post-deploy health check for any service — a bad deploy fails the pipeline instead of failing silently.

Outcome: a working, automatically-deployed prediction endpoint, triggered purely by a Git merge — the core "aha" moment for DevOps engineers.

Day 3 (or a half-day, given "couple of days") — CT loop + CM

1. `monitor.yml`: scheduled GitHub Action that pulls a batch of "new" sample data, runs **Evidently AI** to compare it against the training baseline, and publishes a drift report as a workflow artifact.
2. Lambda logs predictions to **CloudWatch Logs**; add a simple CloudWatch metric filter/alarm for error rate or latency, mirroring what they already do for services.

3. Manually simulate the full loop once: inject drifted data → drift report flags it → manually trigger `train-deploy.yml` → new model deployed. This closes the loop shown in your MLOps lifecycle diagram (Data Exploration → ML → Testing → Ops → back to retrain).

Outcome: the team has walked the entire ring in your reference diagram once, end to end, with tools they can map back to Jenkins/Terraform/Prometheus equivalents later.

6. What the deployed endpoint actually does

`predict.py` is a thin, stateless scoring wrapper around the trained model — nothing more:

1. **Receives** an HTTPS POST with a JSON body containing feature values for one record (e.g. `Contract`, `tenure`, `MonthlyCharges`, `InternetService`).
2. **Loads the model** — baked into the Lambda container image at build time. Warm invocations reuse the in-memory model; a cold start loads it once at container init.
3. **Preprocesses the input** using the same transformations applied at training time (e.g. encoding categorical columns like `Contract/PaymentMethod`, coercing `TotalCharges` to numeric) so raw JSON matches what the model expects.
4. **Runs inference** — `model.predict()` (and `predict_proba()` for a confidence score) on that single row.
5. **Returns JSON**, e.g. `{"prediction": 0, "probability": 0.12}`.
6. **Logs the request/response to CloudWatch** — this feeds the CM step: every prediction becomes a data point to compare against the training distribution later.

It's single-purpose and stateless: one input row → one prediction, no retraining, no memory of prior requests. That's deliberate — it keeps the "model serving" concept simple and keeps Lambda cheap.

7. Testing & consuming the deployed model

Since the endpoint is a Function URL, reaching it is a plain HTTPS POST — no SDK required.

Signed request (AWS_IAM auth), Python example:

```
import boto3
import requests
from botocore.auth import SigV4Auth
from botocore.awsrequest import AWSRequest

url = "https://<your-function-url>.lambda-url.<region>.on.aws/"
payload = '{"Contract": 0, "tenure": 5, "MonthlyCharges": 70.35,
"TotalCharges": 351.75, "InternetService": 1, "PaymentMethod": 2}'

request = AWSRequest(method="POST", url=url, data=payload,
                      headers={"Content-Type": "application/json"})
```

```
SigV4Auth(boto3.Session().get_credentials(), "lambda",
"<region>").add_auth(request)
```

```
resp = requests.post(url, data=payload, headers=dict(request.headers))
print(resp.json())
```

❑ **Local/manual test:** use `awscurl` for a one-liner SigV4-signed curl call during development, instead of hand-rolling the boto3 script each time.

From any other application: the calling app just needs to sign the request with valid AWS credentials (or, if `AuthType: NONE` is used for a quick demo, a plain `curl/requests.post` with no auth) — consuming the model requires nothing MLOps-specific, which is a useful point to make to the team: serving is just an API call.

Automated verification: this same signed-request pattern is what the Day 2 smoke-test step in `train-deploy.yml` uses to confirm a fresh deploy actually works before the pipeline is considered green.

8. Success criteria for the POC

- [] A model is trained, versioned, and tracked (Git + DVC + MLflow)
 - [] CI blocks a bad model/data change automatically
 - [] A merge to `main` deploys a new model to Lambda with no manual steps
 - [] A scheduled job detects data drift and produces a report
 - [] The team can explain, in their own words, what CI/CD/CT/CM mean for ML vs. code
 - [] Total AWS spend stays under \$10 (check billing dashboard on completion)
-

9. After the POC

This POC is intentionally minimal and not meant to graduate into production — that stays with the dedicated ML engineering team. Its purpose is literacy: once DevOps engineers have felt the CT and CM steps hands-on, they're equipped to have informed conversations with the ML team about pipeline ownership boundaries, tooling standards (e.g., should the org standardize on MLflow + DVC, or something heavier like SageMaker Pipelines / Kubeflow later), and where DevOps responsibilities plug into the ML lifecycle long-term.

Cleanup reminder: delete the Lambda function, S3 bucket contents, and the ECR repository after the POC to avoid any lingering (minimal) cost.

10. Phase 2: Going deeper on retraining and drift

The Day 3 loop in the POC (drift report → manual retrain trigger) is a simplified version of a real CT/CM loop. Once the team is comfortable with the basics, these are the concrete next experiments to build maturity — roughly in order of difficulty.

10.1 Retraining (CT) — from manual to automatic

Step	What to try	Why it matters
1. Scheduled retraining	Change <code>train-deploy.yml</code> 's trigger from "on merge" to also run on a <code>cron</code> schedule (e.g. weekly), retraining on whatever data is currently in <code>data/</code> .	Mirrors the most common real-world CT trigger: time-based, not event-based.
2. Data-volume trigger	Add a step that checks how many new rows have landed in S3 since the last training run; only kick off retraining once a threshold is crossed (e.g. 500 new rows).	Avoids retraining on trivial amounts of new data — a real cost/value tradeoff teams have to make.
3. Performance-based trigger	Have <code>monitor.py</code> compare live prediction accuracy (once ground truth is available) against the last recorded training accuracy; if it drops below a threshold, trigger <code>train-deploy.yml</code> via the GitHub API (<code>workflow_dispatch</code>) instead of a person doing it manually.	This is the real "closing the loop" moment — monitoring output directly causes a new training run, no human in the middle.
4. Champion/challenger evaluation	Before promoting a newly retrained model, run it against a held-out validation set alongside the currently deployed ("champion") model. Only promote the new ("challenger") model if it beats the champion on the agreed metric.	Prevents a retrain from silently deploying a worse model — the ML equivalent of a canary/rollback gate.
5. Model registry	Introduce MLflow Model Registry (still free, can run alongside the local MLflow tracking already in place) to formally track model versions and their stage: <code>Staging</code> → <code>Production</code> → <code>Archived</code> .	Gives the team a single source of truth for "what's live right now," instead of inferring it from the latest ECR image tag.

10.2 Drift & monitoring (CM) — from a static report to a real signal

Step	What to try	Why it matters
1. Distinguish drift types	Extend <code>monitor.py</code> 's Evidently AI report to explicitly separate data drift (input feature distributions shifting, e.g. a growing share of customers on Month-to-month contracts vs. what the model was trained on) from prediction drift (the model's output distribution shifting) and — once ground truth is available — concept drift (the relationship between inputs and the true outcome changing).	These require different responses: data drift may just need monitoring, concept drift usually forces a retrain.
2. Ground-truth feedback loop	Simulate a downstream process that eventually reports the real outcome for each prediction (e.g. a delayed CSV of "actually churned" values, since real churn outcomes only become known weeks later). Join this against the logged predictions to compute <i>live</i> accuracy, not just data drift.	Data drift alone doesn't prove the model got worse — this is what actually confirms it.
3. Alerting, not just reporting	Wire the drift report's pass/fail result into a Slack/email notification (SNS + a Lambda, or just a GitHub Actions step that fails the workflow and notifies via a webhook) instead of only producing a report artifact.	Turns monitoring from something someone has to remember to check into something that pages the team.
4. Canary / shadow deployment	Once a challenger model passes champion/challenger evaluation, route a small percentage of live traffic to it (e.g. via a second Lambda alias + weighted routing) before fully promoting it.	Introduces the DevOps-familiar canary release pattern to model deployments specifically.
5. Dashboarding	Push CloudWatch metrics + Evidently AI drift scores into a simple Grafana dashboard (free, can run in a small Docker container) so drift and prediction volume are visible over time, not just per-run.	Gives the team an at-a-glance operational view, same as they'd build for any other service.

10.3 Suggested hands-on exercises (pick 2–3 to start)

- **Inject synthetic drift:** manually skew a batch of "new" input data (e.g. shift the `MonthlyCharges` or `tenure` distribution) and confirm the drift report actually flags it — proves the monitoring is working, not just present.
- **Break the champion/challenger gate on purpose:** intentionally train a worse model (e.g. fewer trees, bad hyperparameters) and confirm the pipeline refuses to promote it.
- **Time the whole loop:** from "drift detected" to "new model live," measure how long the automated loop takes end-to-end, and use that as a baseline to discuss SLAs with the ML team later.
- **Add a rollback drill:** deliberately promote a bad model, then practice rolling the Lambda back to the previous ECR image tag — same muscle as any other production rollback.

These exercises are deliberately still low-cost and low-risk (same \$10-class AWS footprint) — the goal of Phase 2 is depth of understanding of the CT/CM loop, not scaling up infrastructure. Scaling to a real platform (SageMaker Pipelines, Kubeflow, a hosted MLflow/model registry, Airflow orchestration) is a conversation to have with the ML engineering team once this phase is done.